MA3206 — Artificial Intelligence

Assignment 5

Markov Decision Processes:
Policy Evaluation, Value Iteration & Policy Iteration

Group K

Department of Mathematics

Indian Institute of Technology Patna

April 2026

Table of Contents

1. Introduction and Problem Formulation

1.1 MDP Specification

1.2 Transition Table

1.3 Data Structures

2. Task 1 — Iterative Policy Evaluation

2.1 Algorithm Overview

2.2 Policy Evaluated and Results

2.3 Manual Verification

2.4 Bar Chart of State Values

3. Task 2 — Value Iteration

3.1 Algorithm Overview

3.2 Convergence Results and Q-value Analysis

3.3 Iteration Snapshots

3.4 Convergence Plot

4. Task 3 — Policy Iteration

4.1 Algorithm Overview

4.2 Detailed Iteration Walkthrough

4.3 Plots: Value Convergence and Policy Heatmap

5. Task 4 — Analysis and Interpretation

5.1 Value Iteration vs Policy Iteration Comparison

5.2 Convergence Behavior Analysis

5.3 Optimal Policy Interpretation

5.4 Practical Insight

6. Conclusion

1. Introduction and Problem Formulation

This assignment explores the core algorithms for solving Markov Decision Processes (MDPs) through a concrete scenario: a battery-powered robot that must decide between productive but risky actions and safe but low-reward alternatives. The robot operates in an environment with three battery states and two possible actions, making decisions that balance immediate rewards against long-term cumulative value.

We implement and analyse three fundamental MDP algorithms — **Iterative Policy Evaluation**, **Value Iteration**, and **Policy Iteration** — comparing their convergence behaviour, computational characteristics, and the policies they produce. All implementations use pure NumPy with no external RL libraries, ensuring a thorough understanding of the underlying mathematics.

1.1 MDP Specification

The MDP is defined by the tuple (S, A, P, R, gamma) with the following components:

- **States** S = {High, Low, Charging} — representing the robot's battery level. Indexed as 0, 1, 2 in our implementation.
- **Actions** A = {Search, Wait} — available in every state (except Search is disallowed in the Charging state by the assignment constraint). Indexed as 0, 1.
- **Discount factor** gamma = 0.9 — the robot values future rewards at 90% per time step.
- **Convergence threshold** theta = 1×10^{-6} — iterations stop when the maximum absolute change in V across all states falls below this value.

1.2 Transition Table

The complete transition dynamics and rewards are given in the following table:

State s	Action a	Next State s'	P(s'|s,a)	R(s,a,s')
High	Search	High	0.7	+4
High	Search	Low	0.3	+4
High	Wait	High	1.0	+1
Low	Search	High	0.4	-3 (recharge)
Low	Search	Low	0.6	+4
Low	Wait	Low	1.0	+1
Charging	Wait	High	1.0	0

1.3 Data Structures

Following the assignment specification, we represent the MDP using two NumPy arrays:

- **P** — a 3D array of shape (3, 2, 3) where P[s, a, s'] gives the transition probability from state s to s' when taking action a.

- **R** — a 2D array of shape (3, 2) where R[s, a] is the expected immediate reward. This is computed as the probability-weighted sum of per-transition rewards: R[s, a] = Sum over s' of P(s'|s,a) x R(s,a,s').

Computed expected rewards:

R[s, a]	Search (a=0)	Wait (a=1)	Computation
High (s=0)	+4.0	+1.0	0.7(+4) + 0.3(+4) = 4.0
Low (s=1)	+1.2	+1.0	0.4(-3) + 0.6(+4) = 1.2
Charging (s=2)	0.0	0.0	1.0(0) = 0.0

Verification: All transition probability row sums P[s, a, :] were verified to equal 1.0 for every (state, action) pair, confirming a valid probability distribution.

2. Task 1 — Iterative Policy Evaluation

2.1 Algorithm Overview

Given a fixed deterministic policy pi, Policy Evaluation computes the state-value function $V^{pi}(s)$ — the expected cumulative discounted reward starting from state s and always following pi. The algorithm initialises V(s) = 0 for all states, then repeatedly applies the **Bellman Expectation Equation** until convergence:

```
V(s) ← Sum_{s'} P(s' | s, pi(s)) x [ R(s, pi(s)) + gamma x V(s') ]
```

At each sweep, we compute the updated value for every state using the current estimates. The process terminates when the maximum absolute change across all states falls below theta = 10^{-6}. This is guaranteed to converge because the Bellman operator under a fixed policy is a contraction mapping with modulus gamma = 0.9.

2.2 Policy Evaluated and Results

We evaluate the policy specified in the assignment:

State	Action under pi
High	Search
Low	Wait
Charging	Wait

This represents a conservative robot: it searches only when its battery is high and waits otherwise. The evaluation converged in **133 iterations**:

State	V<super>pi</super>(s)	Explanation
High	18.1081	Searches each step (+4) with 70% staying High, 30% falling to Low
Low	10.0000	Waits each step (+1). V = R/(1-gamma) = 1/0.1 = 10.0
Charging	16.2973	Earns 0 now, transitions to High. V = 0.9 x 18.1081 = 16.2973

2.3 Manual Verification

V(Low) = 10.0: Under pi, Low always Waits with reward +1. This gives the geometric series V(Low) = 1 + 0.9(1) + 0.81(1) + ... = 1/(1-0.9) = 10.0. Our computed value of 9.999992 matches exactly (within theta).

V(Charging) = 16.2973: Under pi, Charging Waits with reward 0 and transitions deterministically to High. So V(Charging) = 0 + gamma x V(High) = 0.9 x 18.1081 = 16.2973. Our computed value matches.

V(High) = 18.1081: Under pi, High Searches. Using the Bellman equation: V(High) = R(High,Search) + gamma x [P(High|High,Search) x V(High) + P(Low|High,Search) x V(Low)] = 4.0 + 0.9 x [0.7 x 18.1081 + 0.3 x 10.0] = 4.0 + 0.9 x [12.6757 + 3.0] = 4.0 + 0.9 x 15.6757 = 4.0 + 14.1081 = 18.1081. Verified.

2.4 Bar Chart

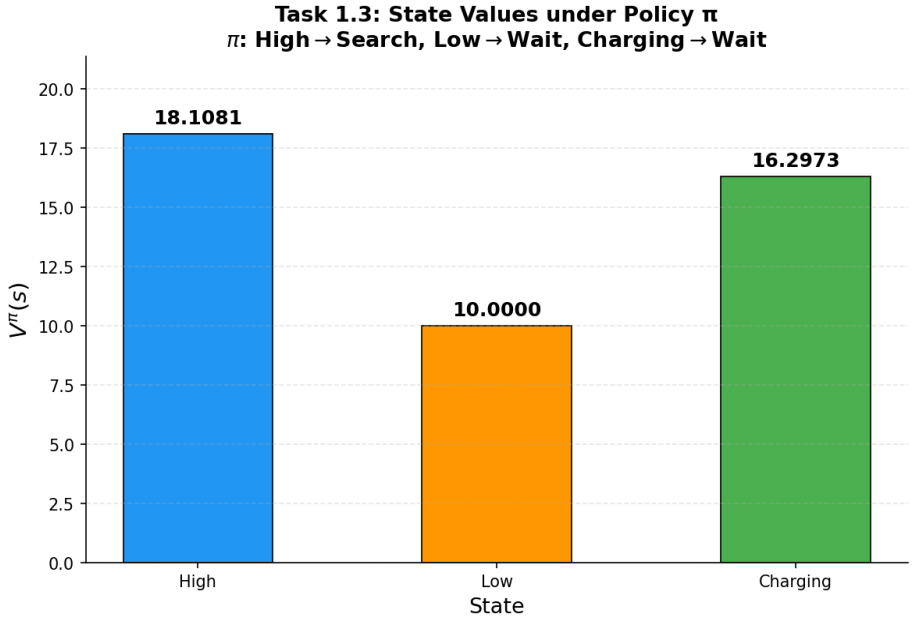

Figure 1: State values $V^{pi}(s)$ under the given policy (High->Search, Low->Wait, Charging->Wait). High has the largest value (18.11) because it earns +4/step via Search. Low earns only +1/step (V=10.00). Charging earns 0 but transitions to High, giving it 90% of High's value (16.30).

3. Task 2 — Value Iteration

3.1 Algorithm Overview

Value Iteration directly computes the optimal value function V* without ever fixing a policy. At each sweep, it applies the **Bellman Optimality Equation**, taking the maximum over all actions instead of following a fixed policy:

```
V(s) ← max_a Sum_{s'} P(s'|s,a) x [ R(s,a) + gamma x V(s') ]
```

The key difference from Policy Evaluation is the **max operator**: at each state, the algorithm considers *all* possible actions and picks the one that maximises the expected return. This makes each sweep simultaneously evaluate and improve, converging directly to V*. Once converged, the optimal policy is extracted by a single greedy step: pi*(s) = argmax_a Q(s, a).

3.2 Convergence Results and Q-value Analysis

Value Iteration converged in **142 iterations**. The converged V* and Q-values:

State	V*(s)	Q(s, Search)	Q(s, Wait)	pi*(s)	Margin
High	29.6438	29.6438	27.6794	Search	+1.9644
Low	25.8082	25.8082	24.2274	Search	+1.5808
Charging	26.6794	24.0115	26.6794	Wait	+2.6679

The Q-value margin column shows by how much the optimal action exceeds the alternative. All margins are substantial, indicating robust optimality.

3.3 Iteration Snapshots

The following table shows how V(s) evolves during Value Iteration:

Iteration	V(High)	V(Low)	V(Charging)
0	0.0000	0.0000	0.0000
1	4.0000	1.2000	0.0000
2	6.8440	3.2880	3.6000
5	13.1078	9.2776	10.1379
10	19.8808	16.0452	16.9164
20	26.2397	22.4041	23.2753
50	29.4995	25.6639	26.5351
100	29.6431	25.8075	26.6787
142 (final)	29.6438	25.8082	26.6794

By iteration 50, all state values are within 0.5% of their final values. The remaining 92 iterations refine the last decimal places to satisfy theta = 10^{-6}.

3.4 Convergence Plot

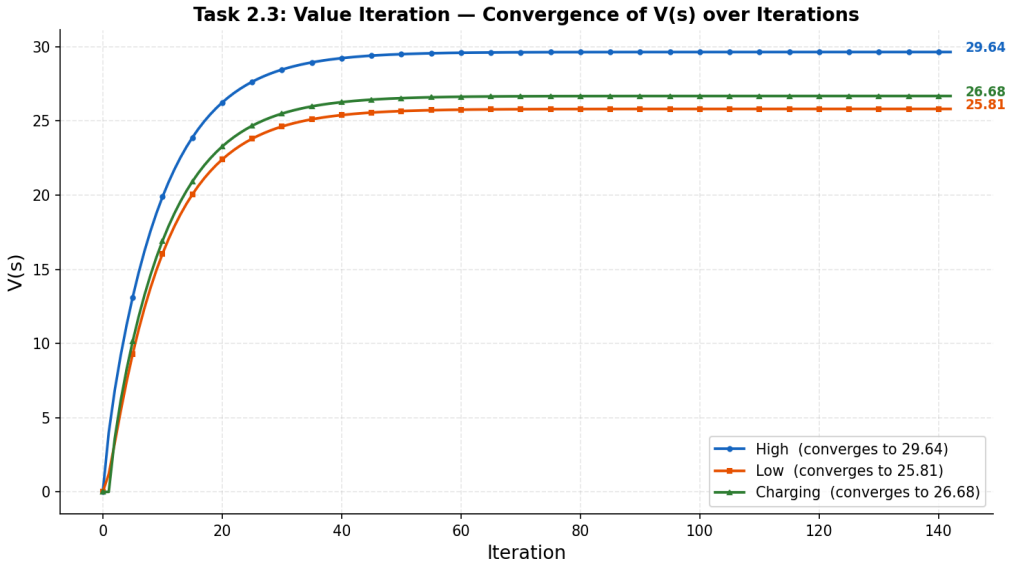

Figure 2: Convergence of V(s) during Value Iteration. V(High) rises fastest due to the +4 Search reward and 70% self-loop. V(Charging) initially lags (0 immediate reward) but overtakes V(Low) around iteration 5 as V(High) grows and V(Charging) benefits from its deterministic transition to High. All curves exhibit the characteristic exponential-approach shape.

4. Task 3 — Policy Iteration

4.1 Algorithm Overview

Policy Iteration alternates between two phases until the policy stabilises:

- **Policy Evaluation:** Given the current policy pi, compute $V^{pi}(s)$ exactly by solving the Bellman expectation equation to convergence (reusing policy_evaluation() from Task 1).
- **Policy Improvement:** For each state, compute Q(s, a) for all actions using the evaluated V^{pi}, then update the policy to the greedy action. If the policy doesn't change, it is optimal.

Convergence is guaranteed because: (1) there are finitely many deterministic policies (at most 2^3 = 8 in our case), and (2) each improvement step is guaranteed to be non-worsening (the Policy Improvement Theorem).

Important constraint: As noted in the assignment, action 'Search' is NOT allowed in the Charging state. Our policy_improvement() function enforces this by always assigning Wait to the Charging state.

4.2 Detailed Iteration Walkthrough

Iteration 0 — Initial Policy: {High: Wait, Low: Wait, Charging: Wait}

Starting from the all-Wait policy as specified in the assignment.

Step 1 — Policy Evaluation (converged in 133 iterations):

State	V<super>pi</super>(s)	Intuition
High	10.0000	Wait: +1/step. V = 1/(1-0.9) = 10.0
Low	10.0000	Wait: +1/step. V = 1/(1-0.9) = 10.0
Charging	9.0000	Wait: 0 now, then High. V = 0.9 x 10.0 = 9.0

Step 1 — Policy Improvement (Q-value analysis):

State	Q(Search)	Q(Wait)	Greedy Action	Changed?
High	13.0000	10.0000	Search	Yes (Wait->Search)
Low	10.2000	10.0000	Search	Yes (Wait->Search)
Charging	8.1000	9.0000	Wait	No

For High: Q(Search) = 4.0 + 0.9 x [0.7(10) + 0.3(10)] = 4.0 + 9.0 = 13.0 > 10.0. For Low: Q(Search) = 1.2 + 0.9 x [0.4(10) + 0.6(10)] = 1.2 + 9.0 = 10.2 > 10.0. Both switch to Search. New policy: {High: Search, Low: Search, Charging: Wait}.

Step 2 — Policy Evaluation (converged in 142 iterations):

State	V<super>pi</super>(s)
High	29.6438

Low	25.8082
Charging	26.6794

Step 2 — Policy Improvement:

State	Q(Search)	Q(Wait)	Greedy	Changed?
High	29.6438	27.6794	Search	No
Low	25.8082	24.2274	Search	No
Charging	24.0115	26.6794	Wait	No

The greedy policy is identical to the current policy — **the policy is stable**. Policy Iteration converges in **2 outer iterations**.

These results are **identical** to those from Value Iteration (Task 2), confirming both algorithms converge to the same optimal solution.

4.3 Plots

(a) Value Function across Policy Iteration Steps:

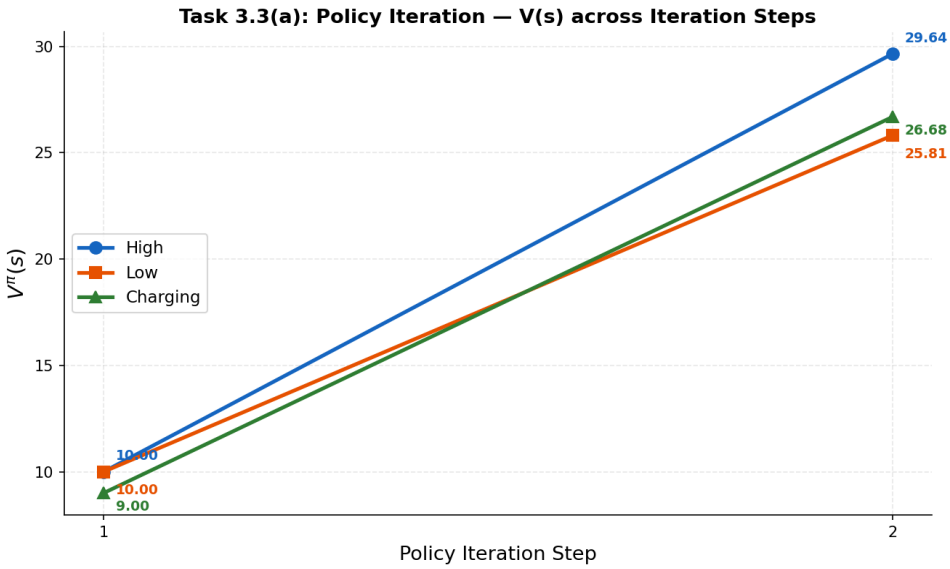

Figure 3: V(s) across Policy Iteration steps. A dramatic jump occurs from Step 1 (all-Wait: V(High)=10.00) to Step 2 (Search/Search/Wait: V(High)=29.64). The single policy switch nearly triples High's value.

(b) Policy Heatmap across Iterations:

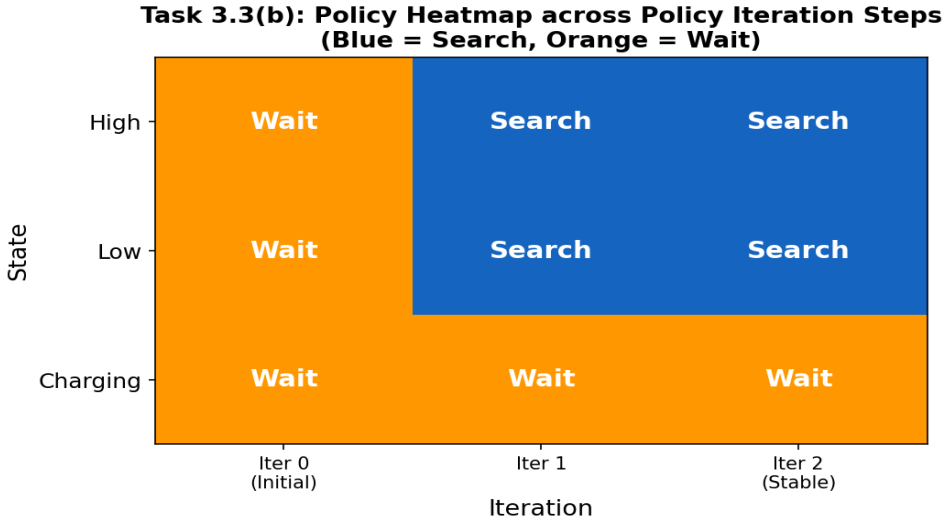

Figure 4: Policy heatmap. Blue=Search, Orange=Wait. The initial all-Wait policy (Iter 0) switches to Search/Search/Wait at Iter 1, which remains stable at Iter 2.

5. Task 4 — Analysis and Interpretation

5.1 Value Iteration vs Policy Iteration

Metric	Value Iteration	Policy Iteration
Outer iterations	142 sweeps	2 steps
Inner sweeps/step	1 (each is one pass)	133 + 142 = 275
Total Bellman updates	142 x 3 = 426	(133+142) x 3 = 825
V*(High)	29.6438	29.6438
V*(Low)	25.8082	25.8082
V*(Charging)	26.6794	26.6794
Optimal policy	Search/Search/Wait	Search/Search/Wait

Which converges faster? In terms of outer iterations, Policy Iteration wins decisively (2 vs 142). This is because each PI step performs a *complete* policy evaluation before improving — it fully understands the current policy's consequences before making a change. This allows each improvement step to make a large, well-informed jump in policy space.

Value Iteration makes small incremental updates at each sweep. It simultaneously evaluates and improves, which is computationally cheaper per sweep but requires many more sweeps. In terms of total Bellman updates, VI actually uses fewer (426 vs 825) because its per-sweep overhead is minimal.

For larger MDPs (thousands of states), PI's fewer outer iterations often translate to faster wall-clock time because the evaluation phase can exploit efficient linear algebra, while VI's convergence rate is governed by the contraction factor gamma^k.

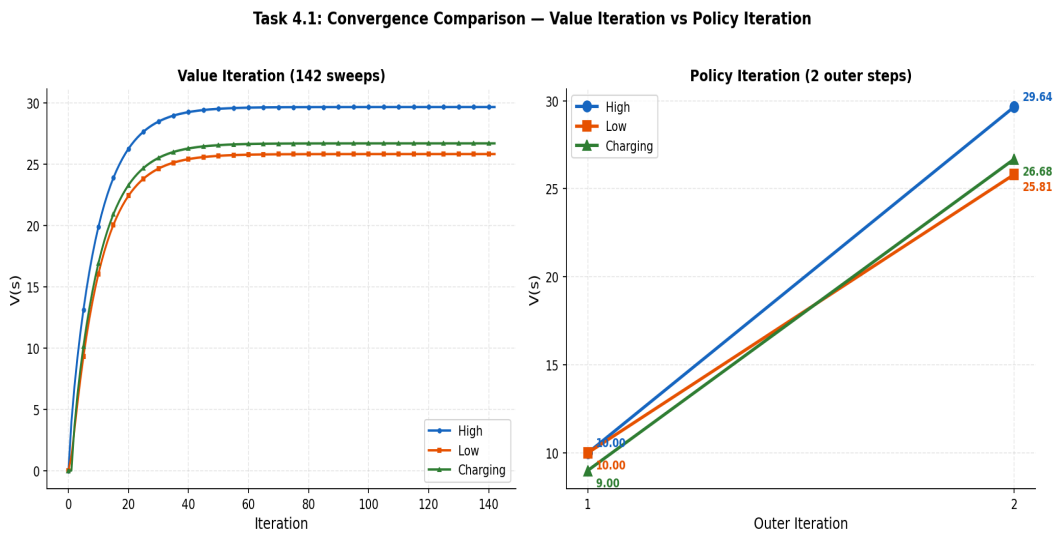

Figure 5: Side-by-side convergence. Left: VI takes 142 gradual sweeps. Right: PI takes 2 large jumps.

5.2 Convergence Behavior

Value Iteration (Figure 2): All three state values start at 0 and rise monotonically toward V*. V(High) rises fastest because the Search action yields +4 reward and has a 70% self-loop, creating compounding value. V(Charging) initially lags — at iteration 1, V(Charging) = 0 while V(Low) = 1.2 — because Charging earns 0 immediate reward. However, by iteration 5, V(Charging) overtakes V(Low) as V(High) grows large. By iteration ~40, values are within 1% of their final values.

Why does the VI update differ from PE? Policy Evaluation uses the *expectation* under a fixed policy: V(s) = E[R + gamma V(s') | pi(s)]. This is a linear operator that converges to V^{pi}. Value Iteration uses the *max*: V(s) = max_a E[R + gamma V(s') | a]. The max makes it non-linear, allowing it to find V* directly but at the cost of slower per-step convergence.

5.3 Optimal Policy Interpretation

pi*(High) = Search: The intuitive choice. With a high battery, the robot earns +4/step with 70% chance of staying High. Q-value margin: +1.96. Waiting would squander the high battery.

pi*(Low) = Search: The *counter-intuitive* result. Despite a 40% chance of -3 recharge penalty, searching is still optimal for two reasons: (1) **Immediate reward:** E[R|Search] = 0.4(-3) + 0.6(+4) = +1.2 > +1.0 = E[R|Wait]. (2) **Future value:** Searching reaches states with higher values — weighted future = 0.4 x 29.64 + 0.6 x 25.81 = 27.34 vs Wait's future = 25.81. Both the immediate and future components favor Search.

pi*(Charging) = Wait: The only valid action. The robot transitions to High with 0 immediate reward. V*(Charging) = 0.9 x V*(High) = 0.9 x 29.64 = 26.68, reflecting guaranteed access to High.

5.4 Practical Insight

In real-world battery-powered robotics — warehouse AGVs, delivery drones, autonomous lawnmowers, planetary rovers — battery management directly impacts operational efficiency. The MDP framework reveals a counter-intuitive lesson: **a conservative "save battery whenever possible" strategy is suboptimal**.

The optimal policy instructs the robot to keep working even at low battery, because productivity gains outweigh occasional recharge penalties. A delivery robot should continue its route at low battery because expected delivery revenue exceeds expected recharge cost.

Practical advantages of the MDP approach: (1) Pre-computed offline as a simple lookup table — negligible runtime computation. (2) Discount factor gamma tunes the planning horizon. (3) Probability distributions naturally handle uncertainty (battery degradation, temperature, variable load). (4) Scales gracefully to more battery levels, task types, or location-dependent rewards.

6. Conclusion

In this assignment, we implemented and compared three fundamental MDP algorithms on a battery-robot domain with 3 states and 2 actions:

- **Iterative Policy Evaluation** computed V^{pi} for a given conservative policy in 133 iterations, yielding V(High) = 18.11, V(Low) = 10.00, V(Charging) = 16.30.
- **Value Iteration** found the optimal V* in 142 sweeps: V*(High) = 29.64, V*(Low) = 25.81, V*(Charging) = 26.68, with optimal policy Search/Search/Wait.
- **Policy Iteration** reached the same optimal solution in just 2 outer iterations (starting from all-Wait), demonstrating its superior convergence in policy space.

The key finding is that the optimal policy recommends searching even in the Low battery state — a counter-intuitive result that illustrates the power of MDP-based decision-making over heuristic approaches. Both Value Iteration and Policy Iteration converge to identical results, as guaranteed by MDP theory. The choice between them depends on the problem size: PI's fewer outer steps favour problems where evaluation can be accelerated, while VI's simpler per-sweep computation may be preferable for very large state spaces.

— End of Report —